МИНИСТЕРСТВО ЦИФРОВОГО РАЗВИТИЯ, СВЯЗИ И МАССОВЫХ КОММУНИКАЦИЙ РОССИЙСКОЙ ФЕДЕРАЦИИ

Ордена Трудового Красного Знамени федеральное государственное бюджетное образовательное учреждение высшего образования
«**Московский технический университет связи и информатики**»
(**МТУСИ**)

Кафедра «Системное программирование»

ОТЧЕТ
по лабораторной работе №4
Вариант №26

по дисциплине «**Параллельные и распределенные вычисления**»

Выполнил:
студент гр. БФИ2202
_____ Сидорук Д. В.
« ____ » _____ 2026 г.

Проверил:
старший преподаватель
_____ Клешнин Н. Г.
ассистент
_____ Соловьев А. С.
« ____ » _____ 2026 г.

Москва, 2026 г.

Содержание

1 Введение 3
2 Цель работы 3
3 Задание 3
4 Ход работы 3
Заключение 8

1 Введение

Обработка изображений требует выполнения однотипных операций над большими массивами данных, что эффективно реализуется на GPU благодаря массовому параллелизму. Однако для достижения высокой производительности важен не только распараллеливание вычислений, но и грамотное управление памятью.

Низкоуровневый API CUDA предоставляет прямой контроль над выделением памяти, передачей данных между хостом и устройством, а также конфигурацией ядер. В отличие от высокоуровневых абстракций, работа с базовыми примитивами позволяет понять архитектуру устройства и оптимизировать доступ к глобальной памяти.

В данной лабораторной работе эти принципы отрабатываются на задаче перевода RGB-изображения в градации серого. Поскольку преобразование каждого пикселя независимо, эта задача позволяет сосредоточиться на корректной организации данных и настройке параметров параллельного выполнения без усложнения логики алгоритма.

2 Цель работы

Изучение алгоритма преобразования цветного изображения в изображение в градациях серого. Получение навыков обработки изображений с помощью CUDA.

3 Задание

Реализовать программу, выполняющую преобразование цветного изображение в изображение в градациях серого с помощью CUDA.

4 Ход работы

В листинге ниже приведено содержание файла rvs_laboratories_3/rvs_laboratories_3.cu, который содержит код, выполняющий преобразование цветного изображения в изображение в градациях серого на GPU с помощью CUDA (лист. 1).

Лист. 1 – Содержание rvs_laboratories_3/rvs_laboratories_3.cu

```
#include "wb.h"

#define wbCheck(stmt) \
  do { \
    cudaError_t err = stmt; \
    if (err != cudaSuccess) { \
      wbLog(ERROR, "Failed to run stmt ", #stmt); \
      wbLog(ERROR, "Got CUDA error ...  ", cudaGetErrorString(err));  \
```

```
      return -1;
        ↪  \
    }
     ↪  \
  } while (0)

//@@ Место для вставки кода
__global__ void grayscale(float *in, float *out, int len) {
  int i = blockIdx.x * blockDim.x + threadIdx.x;

  if (i < len) {
    float r = in[3*i];
    float g = in[3*i + 1];
    float b = in[3*i + 2];

    out[i] = (0.21f * r + 0.71f * g + 0.07f * b);
  }
}


int main(int argc, char *argv[]) {
  wbArg_t args;
  int imageChannels;
  int imageWidth;
  int imageHeight;
  char *inputImageFile;
  wbImage_t inputImage;
  wbImage_t outputImage;
  float *hostInputImageData;
  float *hostOutputImageData;
  float *deviceInputImageData;
  float *deviceOutputImageData;

  args = wbArg_read(argc, argv); /* чтение входных аргументов */

  inputImageFile = wbArg_getInputFile(args, 0);

  inputImage = wbImport(inputImageFile);

  imageWidth  = wbImage_getWidth(inputImage);
  imageHeight = wbImage_getHeight(inputImage);
  // В данной лабораторной значение равно 3
  imageChannels = wbImage_getChannels(inputImage);

  //  Так как изображение монохромное, оно содержит только 1 канал
  outputImage = wbImage_new(imageWidth, imageHeight, 1);

  hostInputImageData  = wbImage_getData(inputImage);
  hostOutputImageData = wbImage_getData(outputImage);

  wbTime_start(GPU, "Doing GPU Computation (memory + compute)");

  wbTime_start(GPU, "Doing GPU memory allocation");
```

```
  wbCheck(cudaMalloc((void **)&deviceInputImageData,
              imageWidth * imageHeight * imageChannels * sizeof(float)));
  wbCheck(cudaMalloc((void **)&deviceOutputImageData,
              imageWidth * imageHeight * sizeof(float)));
  wbCheck(cudaMemset(deviceOutputImageData, 0, imageWidth * imageHeight * sizeof(float)));
  wbTime_stop(GPU, "Doing GPU memory allocation");

  wbTime_start(Copy, "Copying data to the GPU");
  wbCheck(cudaMemcpy(deviceInputImageData, hostInputImageData,
              imageWidth * imageHeight * imageChannels * sizeof(float),
              cudaMemcpyHostToDevice));
  wbTime_stop(Copy, "Copying data to the GPU");

  ////////////////////////////////////////////////////////
  wbTime_start(Compute, "Doing the computation on the GPU");

  auto inputLength = imageWidth * imageHeight;
  int blockSize = 256;
  int gridSize = (inputLength + blockSize - 1) / blockSize;
  grayscale<<<gridSize, blockSize>>>(deviceInputImageData, deviceOutputImageData, inputLength);
  wbCheck(cudaDeviceSynchronize());
  wbCheck(cudaGetLastError());
  //@@ Место для вставки кода

  wbTime_stop(Compute, "Doing the computation on the GPU");

  ////////////////////////////////////////////////////////
  wbTime_start(Copy, "Copying data from the GPU");
  wbCheck(cudaMemcpy(hostOutputImageData, deviceOutputImageData,
              imageWidth * imageHeight * sizeof(float),
              cudaMemcpyDeviceToHost));
  wbTime_stop(Copy, "Copying data from the GPU");

  wbTime_stop(GPU, "Doing GPU Computation (memory + compute)");

  wbSolution(args, outputImage);

  wbCheck(cudaFree(deviceInputImageData));
  wbCheck(cudaFree(deviceOutputImageData));

  wbImage_delete(outputImage);
  wbImage_delete(inputImage);

  return 0;
}
```

В листинге ниже приведено содержание файла CMakeLists.txt, определяющего правила сборки программы (лист. 2).

Лист. 2 – Содержание CMakeLists.txt

```
project(rvs_laboratories_3)

find_package(CUDA REQUIRED)

add_executable(dataset_generator
 ↪  dataset_generator/dataset_generator.cpp)
cuda_add_executable(rvs_laboratories_3
 ↪  rvs_laboratories_3/rvs_laboratories_3.cu)
```

В листинге ниже приведен результат сборки и выполнения программы (лист. 3)

Лист. 3 – Результат работы программы

```
eoanermine@eoanermine:~/Repositories/rvs_laboratories_3/build$
 ↪  cmake ..
-- The C compiler identification is GNU 13.3.0
-- The CXX compiler identification is GNU 13.3.0
-- Detecting C compiler ABI info
-- Detecting C compiler ABI info - done
-- Check for working C compiler: /usr/bin/cc - skipped
-- Detecting C compile features
-- Detecting C compile features - done
-- Detecting CXX compiler ABI info
-- Detecting CXX compiler ABI info - done
-- Check for working CXX compiler: /usr/bin/c++ - skipped
-- Detecting CXX compile features
-- Detecting CXX compile features - done
-- Performing Test CMAKE_HAVE_LIBC_PTHREAD
-- Performing Test CMAKE_HAVE_LIBC_PTHREAD - Success
-- Found Threads: TRUE
-- Found CUDA: /usr/local/cuda (found version "13.1")
-- Configuring done (4.2s)
-- Generating done (0.1s)
-- Build files have been written to:
 ↪  ~/Repositories/rvs_laboratories_3/build
eoanermine@eoanermine:~/Repositories/rvs_laboratories_3/build$
 ↪  make
[ 50%] Building NVCC (Device) object CMakeFiles/rvs_laboratories_3
 ↪  .dir/rvs_laboratories_3/rvs_laboratories_3_generated_main.cu.o
[100%] Linking CXX executable rvs_laboratories_3
[100%] Built target rvs_laboratories_3
eoanermine@eoanermine:~/Repositories/rvs_laboratories_3/build$
 ↪  ./rvs_laboratories_3 input.ppm output.pbm
[GPU    ] 0.000130816 Doing GPU memory allocation
[Copy   ] 0.000020992 Copying data to the GPU
[Compute] 0.014000896 Doing the computation on the GPU
```

```
[Copy   ] 0.000029952 Copying data from the GPU
[GPU    ] 0.014286848 Doing GPU Computation (memory + compute)
Solution is correct.
```

На рисунке ниже приведено исходное изображение (рис. 1).

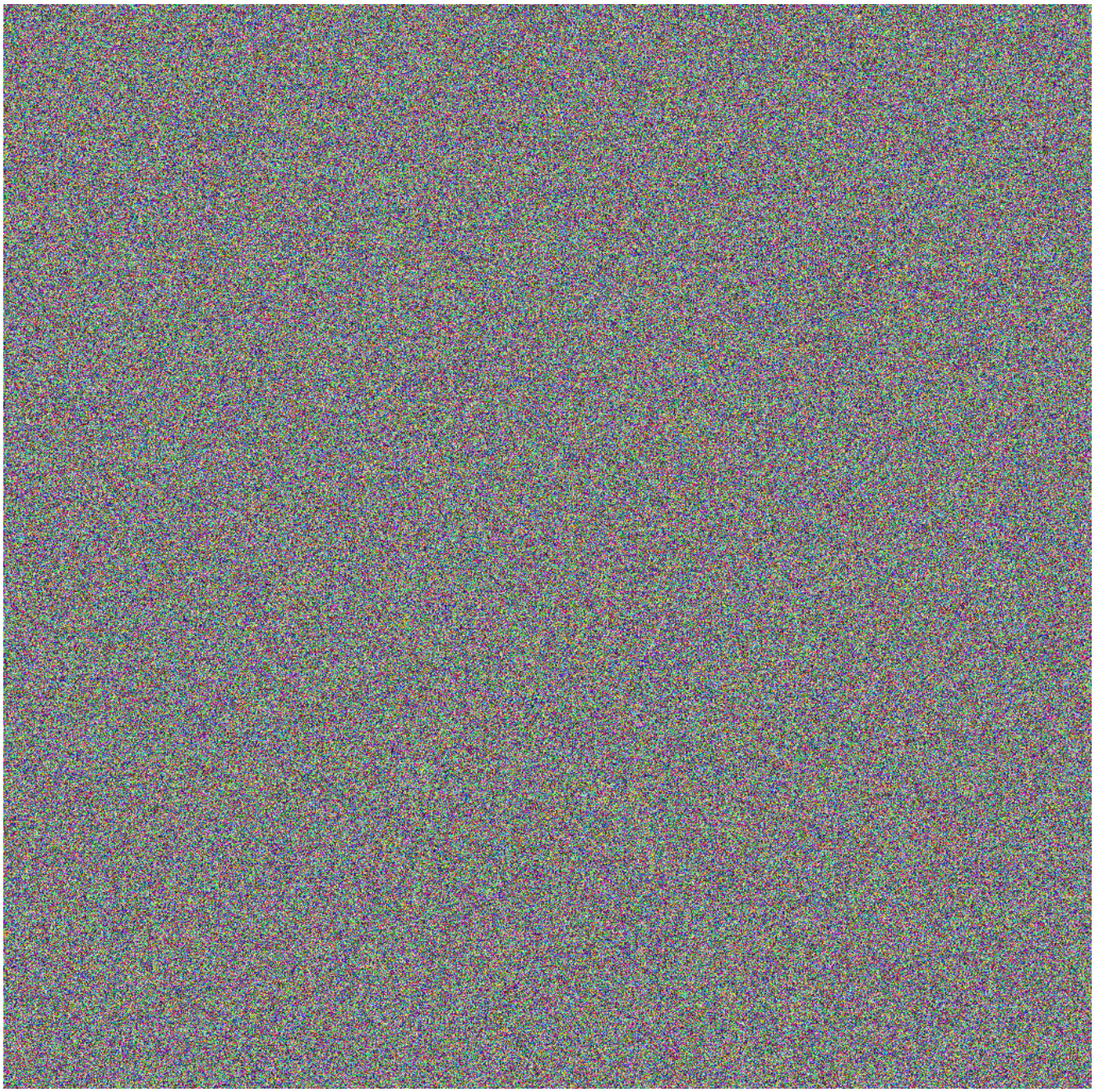

Рис. 1 – Исходное изображение

На рисунке ниже приведено изображение, полученное в результате работы программы (рис. 2).

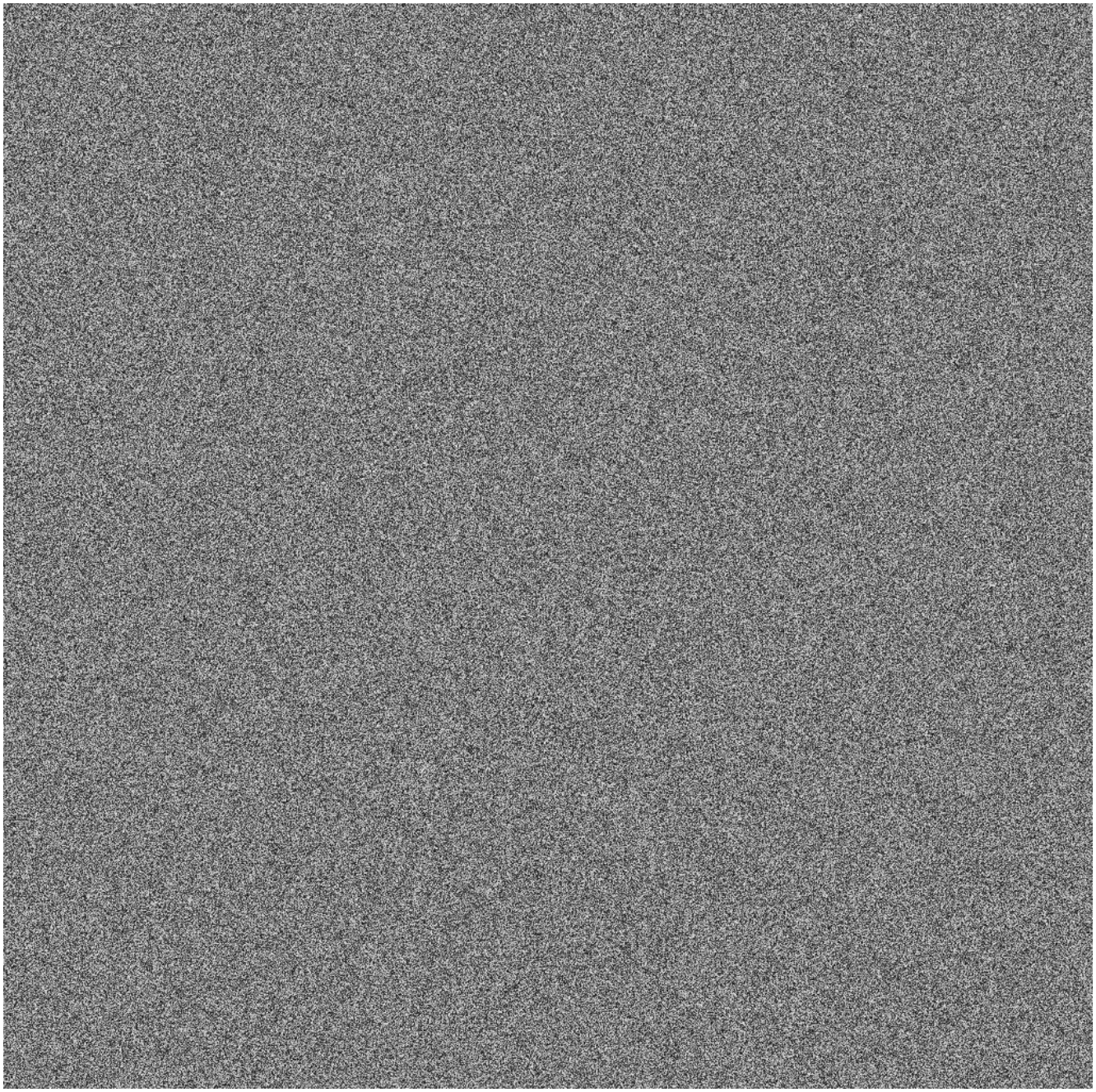

Рис. 2 – Изображение, полученное в результате работы программы

Как видно, изображение было успешно переведено из цветного в изображение в градациях серого.

Заключение

Таким образом, в ходе выполнения данной лабораторной работы был успешно изучен алгоритм преобразования цветного изображение в изображение в градациях серого. Были получены навыки обработки изображений с помощью CUDA. На практике была реализована программа, преобразующая цветное изображение в изображение в градациях серого с помощью CUDA.